

详细设计说明书

1. 分析聚合器设计

聚合器负责调用各维度分析器 并将结果统一转成前端可展示的数据结构

核心输出

```
interface AnalysisData {  
  repo_url: string;  
  health_score: number;  
  badge_level: string;  
  badge_color: string;  
  badge_description: string;  
  dimensions: AnalysisDimension[];  
  ai_diagnosis: AIDiagnosisItem[];  
  analyzed_at: string;  
}
```

评分策略

1. 每个维度输出 0-100 分
2. 聚合器按维度平均或权重计算总分
3. 总分映射到 Badge 等级
4. 诊断建议按严重程度排序

2. 六维分析器设计

2.1 代码质量分析器

输入 仓库路径

处理

- 识别 Python JavaScript TypeScript Go 等语言文件
- 使用 radon/lizard 统计复杂度
- 统计语言分布 代码规模和高复杂函数

输出

- score
- issues
- details.language_breakdown

2.2 测试覆盖分析器

处理

- 检测 `tests/ __tests__/ *.test.* *.spec.*`
- 检测 `pytest Vitest Jest` 等配置
- 检测 CI 中是否执行测试

2.3 架构健康分析器

处理

- 检查目录分层是否清晰
- 识别过大文件和模块
- 检查循环依赖或高耦合风险

2.4 文档完整性分析器

处理

- 检查 `README CONTRIBUTING SECURITY CHANGELOG LICENSE`
- 检查 `.env.example` 是否覆盖实际环境变量
- 检查 `API/部署/测试说明`

2.5 依赖安全分析器

处理

- 检查 `requirements package.json lock` 文件
- 使用 `bandit/pip-audit` 等工具识别安全问题
- 对缺少锁文件或依赖过旧给出建议

2.6 工程规范分析器

处理

- 检查 `CI/CD lint format` 测试脚本
- 检查 `EditorConfig Prettier Black ESLint` 等配置
- 检查 `Docker` 启动脚本和部署说明

3. 缓存设计

缓存键 规范化后的仓库 URL hash

缓存行为

- 成功结果保存 30 分钟

- 缓存命中直接返回
- 错误结果不长期阻塞 允许重新分析
- 返回缓存结果时使用深拷贝 避免外部修改内部状态

4. 异步任务设计

后端使用 `asyncio.create_task` 创建后台分析任务

任务状态

- `pending` 任务正在分析
- `completed` 缓存中存在成功结果
- `failed` 缓存中存在错误结果
- `not_found` 任务不存在或已过期

竞态保护

- `_pending_tasks` 通过 `threading.Lock` 保护

5. AI 诊断设计

诊断输入

- 六维分析结果
- 仓库 URL
- 可选 API Key

诊断输出

```
interface AIDiagnosisItem {  
    advice: string;  
    severity: 'high' | 'medium' | 'low';  
    estimated_hours: number;  
    confidence: number;  
    need_human_review: boolean;  
}
```

降级策略

- 用户未配置 Key 本地规则诊断
- LLM 请求失败 回退本地规则
- 缓存无诊断且用户请求诊断 补充 AI 诊断并更新缓存

6. 前端交互设计

6.1 首页

首页使用高级视觉组件

- Iridescence WebGL 背景
- CurvedLoop 标题
- GlassCard 输入区
- Dock 底部导航

6.2 报告页

状态

- loading 分步进度条
- success 报告内容
- error 错误卡片

核心交互

- 导出 HTML
- 打印 PDF
- 生成分享卡片
- 复制 Badge
- 跳转对比页并预填仓库 A

6.3 对比页

校验

- A/B 都必须是 GitHub URL
- A/B 不能相同

展示

- A / VS / B 三列布局
- PK Arena
- 雷达图和维度差异
- PK 海报生成

7. 异常处理

场景	处理方式
URL 非法	前端提示错误

场景	处理方式
克隆失败	后端返回失败结果 前端展示错误
分析超时	返回 timeout 并清理临时目录
LLM 失败	回退本地诊断
Badge 数据缺失	返回灰色未知 Badge
导出弹窗被拦截	前端提示允许弹窗